INF398 - Introducción al aprendizaje automático

Physics-informed neural networks (PINNs)

Alejandro Veloz

Outline

The problem: learning functions with limited data.

The prior: physical laws as differential equations.

The model: a neural network as a differentiable function.

The key idea: residuals as training losses.

Autodiff: derivatives of the neural network.

Example: damped oscillator.

Limitations

The basic problem

We often want to learn a function

Many scientific problems ask for an unknown function:

$$u = u(x, t)$$

Examples:

- displacement over time,
- temperature over space and time,
- pressure or velocity fields,
- concentration in a reaction-diffusion system,
- voltage or current in a dynamical system.

The function is the object we want to approximate.

Data alone may be insufficient

A purely data-driven model learns from observed pairs:

$$(x_i, t_i) \longrightarrow u_i$$

But data may be:

- sparse,
- noisy,
- expensive,
- irregularly sampled,
- unavailable in parts of the domain.

If data are scarce, we need other sources of information.

Physics gives structure

Many systems obey equations such as:

$$\mathcal{N}[u](x,t) = 0$$

where $\mathcal{N}$ is a differential operator.

This equation tells us which functions are physically plausible.

The equation is a prior over possible functions.

Classical numerical view

Traditional numerical solvers approximate u on a mesh:

$$u(x,t) \approx u_j^n$$

Then derivatives are approximated by finite differences, finite elements, finite volumes, etc.

PINNs take a different view:

$$u(x,t) \approx u_\theta(x,t)$$

where u_θ is a neural network.

The PINN idea

A neural network as a function approximator

A neural network can represent a differentiable function:

$$u_\theta(x, t)$$

Inputs: (x, t)

Output: predicted field value u

For an ODE, the input may be only time t.

What makes a PINN different?

A standard ANN minimizes data error:

$$\mathcal{L}_{\text{data}} = \frac{1}{N_d} \sum_i \| u_\theta(x_i, t_i) - u_i \|^2$$

A PINN also minimizes equation error:

$$\mathcal{L}_{\text{physics}} = \frac{1}{N_f} \sum_j \| \mathcal{N}[u_\theta](x_j, t_j) \|^2$$

PINNs train on data points and physics points.

Physics residual

Define the residual:

$$r_\theta(x,t) = \mathcal{N}[u_\theta](x,t)$$

If u_θ satisfies the governing equation:

$$r_\theta(x,t) = 0$$

So we train the network to make:

$$r_\theta(x,t) \approx 0$$

at selected collocation points.

Collocation points

Collocation points are points where we enforce the equation:

$$\{(x_j, t_j)\}_{j=1}^{N_f}$$

They do not need measured labels.

At each collocation point we ask:

Does the predicted function satisfy the equation here?

Complete PINN loss

Most PINN losses have this structure:

$$\mathcal{L} = \lambda_d \mathcal{L}_{\text{data}} + \lambda_b \mathcal{L}_{\text{boundary}} + \lambda_i \mathcal{L}_{\text{initial}} + \lambda_f \mathcal{L}_{\text{physics}}$$

Each term enforces a different requirement.

The weights λ determine the trade-off between requirements.

Boundary and initial conditions

Differential equations need extra information.

Initial condition:

$$u(x, 0) = u_0(x)$$

Boundary condition:

$$u(x, t) = g(x, t), \qquad x \in \partial\Omega$$

PINNs usually enforce these with additional loss terms.

Autodiff makes PINNs practical

Derivatives of the neural network

To evaluate $\mathcal{N}[u_\theta]$, we need derivatives:

$$\frac{\partial u_\theta}{\partial t}, \qquad \frac{\partial u_\theta}{\partial x}, \qquad \frac{\partial^2 u_\theta}{\partial x^2}.$$

Automatic differentiation computes these derivatives exactly for the computational graph.

No finite-difference stencil is needed.

Autodiff sketch

```
def grad(y, x):
    return torch.autograd.grad(
        y, x,
        grad_outputs=torch.ones_like(y),
        create_graph=True
    )[0]

u = model(t)
u_t = grad(u, t)
u_tt = grad(u_t, t)
```

`create_graph=True` allows higher-order derivatives.

The training loop

At each iteration:

1. sample data points,
2. sample collocation points,
3. compute neural network predictions,
4. compute derivatives with autodiff,
5. compute data, boundary, initial, and physics losses,
6. update parameters with backpropagation.

PINNs still use ordinary gradient-based optimization.

Example: damped oscillator

Governing equation

Consider a damped oscillator:

$$x''(t) + 2\zeta\omega_n x'(t) + \omega_n^2 x(t) = 0$$

with:

$$x(0) = x_0, \qquad x'(0) = v_0$$

The neural network approximation is:

$$x_\theta(t) \approx x(t)$$

Neural surrogate

The network maps:

$$t \longrightarrow x_\theta(t)$$

Example:

$$1 \to 32 \to 32 \to 32 \to 1$$

Because we need derivatives, smooth activations such as tanh are common.

Residual for this ODE

Compute:

$$x'_\theta(t), \qquad x''_\theta(t)$$

Then:

$$r_\theta(t) = x''_\theta(t) + 2\zeta\omega_n x'_\theta(t) + \omega_n^2 x_\theta(t)$$

Physics loss:

$$\mathcal{L}_{\text{physics}} = \frac{1}{N_f}\sum_j \|r_\theta(t_j)\|^2$$

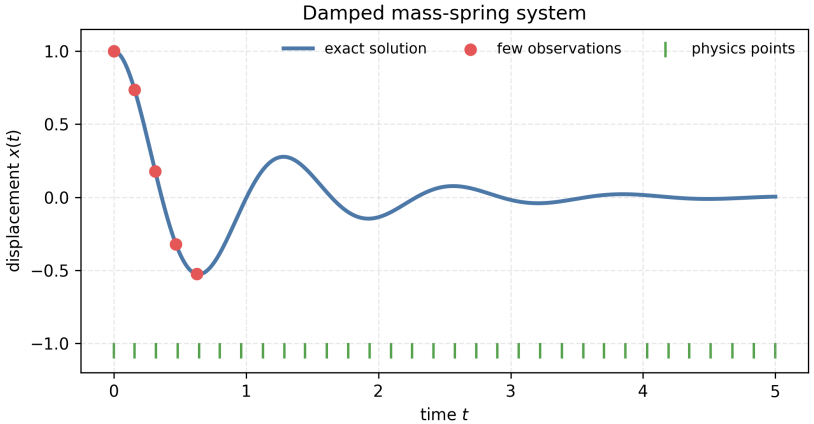
Initial condition loss

The initial condition loss is:

$$\mathcal{L}_{\text{IC}} = \|x_\theta(0) - x_0\|^2 + \|x'_\theta(0) - v_0\|^2$$

This selects the correct trajectory among all solutions of the ODE.

Data plus physics



Red points: sparse observations. Green marks: collocation points for the equation.

Full loss

For the oscillator:

$$\mathcal{L} = \lambda_d \mathcal{L}_{\text{data}} + \lambda_i \mathcal{L}_{\text{IC}} + \lambda_f \mathcal{L}_{\text{physics}}$$

where:

$$\mathcal{L}_{\text{data}} = \frac{1}{N_d} \sum_k \| x_\theta(t_k) - x_k \|^2$$

The same function must fit observations and obey the ODE.

Forward and inverse problems

Forward problem

Known:

- equation,
- physical parameters,
- initial/boundary conditions.

Unknown:

- solution $u(x, t)$.

The PINN learns the solution function:

$$u_\theta(x, t)$$

Inverse problem

Known:

- equation structure,
- partial observations.

Unknown:

- solution,
- one or more physical parameters.

Example:

$$x'' + 2\zeta\omega_n x' + \omega_n^2 x = 0$$

where ζ is unknown and learned from data.

Why inverse problems are hard

Parameter identification depends on:

- data coverage,
- noise level,
- parameter identifiability,
- loss weights,
- optimization quality,
- correctness of the assumed equation.

Physics constrains the problem, but does not remove all ambiguity.

Example notebook

The notebook implements:

- exact damped oscillator solution,
- sparse observations,
- data-only ANN,
- PINN with physics residual,
- comparison of extrapolation behavior,
- inverse problem extension.

Strengths and limitations

Why PINNs are useful

PINNs are useful when:

- data are limited,
- measurements are noisy,
- equations are trusted,
- samples are irregular,
- inverse problems matter,
- differentiable surrogates are useful.

They combine learning with scientific structure.

PINNs are not magic

Common difficulties:

- balancing loss terms,
- stiffness and multiscale dynamics,
- discontinuities or shocks,
- poor collocation sampling,
- bad nondimensionalization,
- wrong or incomplete physics,
- local minima and optimization issues.

Physics-informed does not mean physics-guaranteed.

Practical advice

Start simple:

- solve an ODE before a PDE,
- nondimensionalize variables,
- normalize inputs,
- monitor loss terms separately,
- compare with known solutions,
- vary collocation points,
- test sensitivity to loss weights.

Summary

- A PINN represents an unknown function with a neural network.
- Physics enters through differential equation residuals.
- Autodiff computes derivatives of the neural surrogate.
- The loss combines data, boundary/initial conditions, and physics.
- PINNs can solve forward and inverse problems.
- Training requires careful scaling, sampling, and diagnostics.